



Mata Kuliah : Pengolahan Citra dan Visi Komputer (PCVK)
Program Studi : D4 – Teknik Informatika
Semester : 5

Kelas : TI-3C
NIM : 244107020086
Nama : Zacky Rio Orlando
Jobsheet Ke- : 3

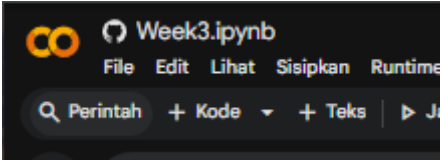
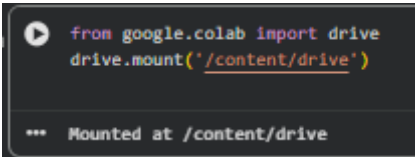
Laporan Jobsheet

Link Google Colab:

<https://colab.research.google.com/drive/1gX9WCHjnh07tRnWjsvXvKL6ncaR5wD4w?usp=sharing>

D. PRAKTIKUM

D1. Operasi Citra Sederhana

Langkah	Jawaban/Deskripsi
1	Buka https://colab.research.google.com/, pilih tab Github dan pastikan repository yang terpilih ada repository yang sama dengan praktikum pada minggu pertama dan kedua. Lanjutkan dengan membuat notebook baru dan ubah nama file menjadi “Week3.ipynb”. Perhatian: Jangan lupa untuk menyimpan salinan ke Github setelah melakukan perubahan / ketika Anda sudah selesai melakukan praktikum. 
2	Akses folder images pada Google Drive Anda dengan kode berikut:  Ikuti alur autorisasinya hingga muncul pesan: “Mounted at /content/drive”.



3

Melakukan transformasi linier brightness dengan memasukkan nilai konstanta tertentu dan menghasilkan warna. Seperti yang telah dibahas pada ulasan teori, formula untuk melakukan transformasi linier brightness adalah sebagai berikut:

$$g(x, y) = f(x, y) + b$$

dimana $g(x,y)$ adalah nilai pixel setelah transformasi, $f(x,y)$ adalah nilai pixel asli, dan b adalah nilai brightness.

Tuliskan potongan kode berikut untuk mengimplementasikan linier brightness pada google colab:

```
print(' Mengubah tingkat kecerahan citra ')
print('-----')
try:
    brightness = int(input('Masukkan nilai kecerahan: '))
except ValueError:
    print('Error, not a number')

#sesuaikan dengan file dan folder di drive Anda
original = cv.imread('/content/drive/MyDrive/PCVK 2026/Gambar/rumah.jpeg')
brightness_image = np.zeros(original.shape, original.dtype)

#akses per piksel
#np.clip digunakan untuk melakukan truncate pixel (nilai dibatasi antara 0-255)
for y in range(original.shape[0]):
    for x in range(original.shape[1]):
        for c in range(original.shape[2]):
            brightness_image[y,x,c] = np.clip(original[y,x,c] + brightness, 0, 255)

#cara simple tanpa for loop
#brightness_image = cv.convertScaleAbs(original, beta=brightness)

final_frame = cv.hconcat((original, brightness_image))
cv2.imshow('final_frame')
```

Dari potongan kode di atas dapat dilihat bahwa ketika dijalankan, system akan menampilkan *text field* untuk memasukkan konstanta nilai *brightness* yang diinginkan, dan akan disimpan pada variabel *brightness*. Kemudian ditentukan citra pada drive yang akan diolah dan ditampung di variabel *original*. Tahap selanjutnya adalah mengakses pixel citra masukan dengan 3 perulangan. Perulangan pertama dilakukan pada *shape[0]* untuk tinggi citra, perulangan kedua dilakukan pada *shape[1]* untuk lebar citra, dan perulangan ketiga dilakukan pada *shape[2]* yang merupakan channel warna pada citra. Setelah melakukan 3 perulangan, dilakukan transformasi linier brightness dengan cara menambahkan nilai brightness pada citra masukan. Contoh hasil dari kode program di atas adalah sebagai berikut.



TUGAS PRAKTIKUM

Langkah	Jawaban/Deskripsi
1	Implementasikan inverse citra pada Google Colaboratory menggunakan formula yang terdapat pada bagian Ulasan Teori, sehingga menghasilkan keluaran seperti berikut: <pre> # Membaca citra original = cv.imread('/content/drive/MyDrive/PCVK 2026/Gambar/buah.png') # Membuat holder untuk citra inverse inverse_image = np.zeros(original.shape, original.dtype) # Proses inverse setiap piksel for y in range(original.shape[0]): for x in range(original.shape[1]): for c in range(original.shape[2]): inverse_image[y, x, c] = 255 - original[y, x, c] # Menggabungkan citra asli dan inverse final_frame = cv.hconcat((original, inverse_image)) # Menampilkan hasil cv2.imshow('final_frame') </pre> 



2	Implementasikan transformasi contrast pada Google Colaboratory menggunakan formula yang terdapat pada bagian Ulasan Teori untuk kontras, sehingga menghasilkan keluaran seperti berikut: <pre>print('Mengubah kontras dan tingkat kecerahan citra.') print('-----') try: brightness = int(input('Masukkan tingkat kecerahan: ')) contrast = float(input('Masukkan kontras: ')) except ValueError: print('Error, not a number') # Sesuaikan dengan file dan folder di Google Drive kamu original = cv.imread('/content/drive/MyDrive/KVM 2826/Gambar/Bangunan.png') contrast_brightness_image = np.zeros(original.shape, original.dtype) # Akses per-piksel dengan formula kontras (alpha) dan kecerahan (beta) # np.clip digunakan untuk membatasi nilai piksel agar tetap berada di antara 0-255 for y in range(original.shape[0]): for x in range(original.shape[1]): for c in range(original.shape[2]): val = contrast * int(original[y, x, c]) + brightness contrast_brightness_image[y, x, c] = np.clip(val, 0, 255) # Cara simpel tanpa for loop (alternatif fungsi bawaan OpenCV): # contrast_brightness_image = cv.cvtColor(original, alpha=contrast, beta=brightness) final_frame = cv.hstack([original, contrast_brightness_image]) cv2.imshow('final_frame')</pre> == Mengubah kontras dan tingkat kecerahan citra ----- Masukkan tingkat kecerahan: 10 Masukkan kontras: 1.5 
3	Implementasikan transformasi logarithmic brightness pada Google Colaboratory menggunakan formula yang terdapat pada bagian Ulasan Teori untuk transformasi log, sehingga menghasilkan keluaran seperti berikut:



	<pre>print(" Mengubah tingkat kecerahan citra dengan Transformasi Log ") print("-----") try: # Nilai ini berfungsi sebagai konstanta pengali (c) dalam formula log c = float(input("Masukkan nilai kecerahan: ")) except ValueError: print("Error, not a number") # Sesuaikan dengan file dan folder di Google Drive kamu original = cv.imread('/content/drive/MyDrive/PCW 2026/Gambar/rumah.jpg') # Pastikan citra bertipe float untuk kalkulasi matematika logaritmik yang akurat original_float = original.astype(np.float32) # Menetapkan formula transformasi logaritmik: s = c * log(1 + r) # np.logip menghitung log(1 + x) yang lebih presisi untuk nilai kecil log_image = c * np.logip(original_float) # Normalisasi kembali ke rentang 0-255 dan ubah tipe data menjadi uint8 log_image = np.clip(log_image, 0, 255).astype(np.uint8) # Menggabungkan gambar asli dan hasil transformasi secara berdampingan (horizontal) final_frame = cv.hconcat((original, log_image)) cv2.imshow('final_frame')</pre> --- Mengubah tingkat kecerahan citra dengan Transformasi Log ----- Masukkan nilai kecerahan: 50
4	Implementasikan transformasi grayscale menggunakan metode averaging, lightness, dan luminance pada Google Colaboratory menggunakan formula yang terdapat pada bagian Ulasan Teori, sehingga menghasilkan keluaran seperti berikut: - Averaging - Lightness - Luminance



```
# 1. Load gambar
img_bgr = cv.imread('/content/drive/MyDrive/PCVK 2026/Gambar/buah.png')

# OpenCV memuat gambar dalam format BGR, konversi ke RGB untuk ditampilkan dengan benar lewat matplotlib
img_rgb = cv.cvtColor(img_bgr, cv.COLOR_BGR2RGB)

# Ambil ukuran dimensi citra
height, width, _ = img_rgb.shape

# Inisialisasi citra kosong untuk masing-masing metode
img_averaging = np.zeros((height, width), dtype=np.uint8)
img_lightness = np.zeros((height, width), dtype=np.uint8)
img_luminance = np.zeros((height, width), dtype=np.uint8)

# Proses transformasi piksel per piksel
for y in range(height):
    for x in range(width):
        r = int(img_rgb[y, x, 0])
        g = int(img_rgb[y, x, 1])
        b = int(img_rgb[y, x, 2])

        # a. Metode Averaging: Rata-rata dari R, G, dan B
        img_averaging[y, x] = np.uint8((r + g + b) / 3)

        # b. Metode Lightness: Rata-rata dari nilai maksimum dan minimum komponen R, G, B
        max_val = max(r, g, b)
        min_val = min(r, g, b)
        img_lightness[y, x] = np.uint8((max_val + min_val) / 2)

        # c. Metode Luminance: Rata-rata tertimbang berdasarkan persepsi mata manusia
        img_luminance[y, x] = np.uint8(0.299 * r + 0.587 * g + 0.114 * b)

# 2. Plotting menggunakan Matplotlib
plt.figure(figsize=(12, 10))
```

```
# 2. Plotting menggunakan Matplotlib
plt.figure(figsize=(12, 10))

# --- a. Averaging ---
plt.subplot(3, 2, 1)
plt.title("a. Averaging (Original)")
plt.imshow(img_rgb)

plt.subplot(3, 2, 2)
plt.title("a. Averaging (Grayscale)")
plt.imshow(img_averaging, cmap='gray', vmin=0, vmax=255)

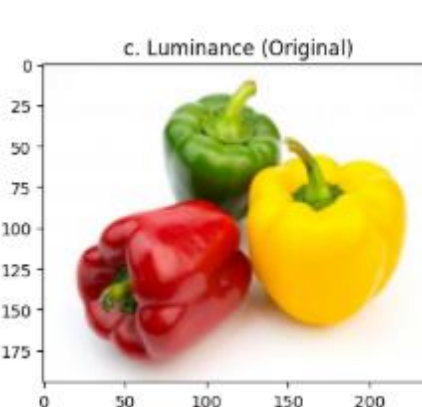
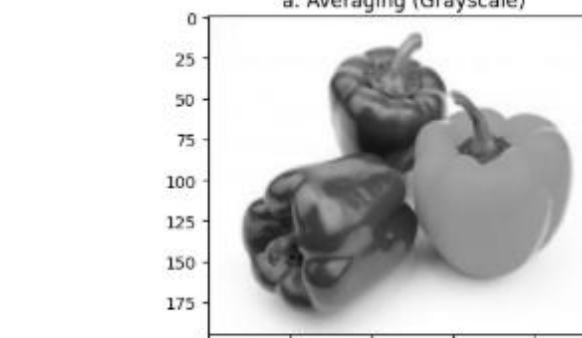
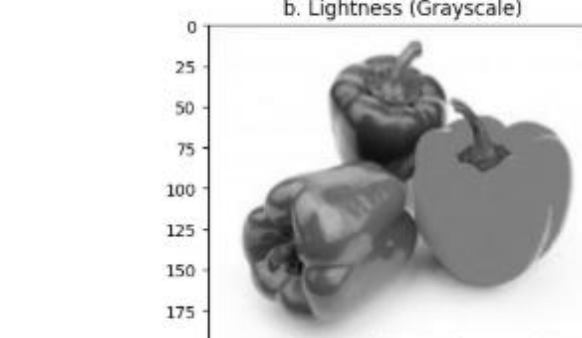
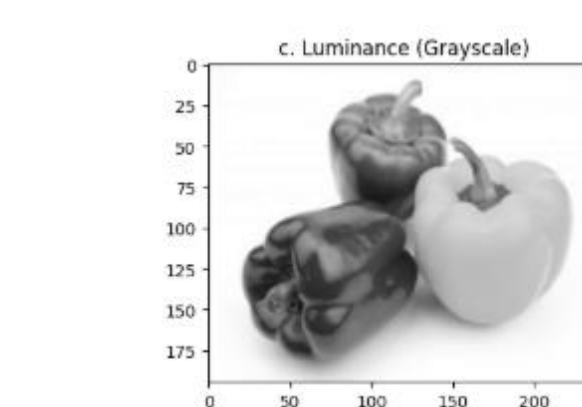
# --- b. Lightness ---
plt.subplot(3, 2, 3)
plt.title("b. Lightness (Original)")
plt.imshow(img_rgb)

plt.subplot(3, 2, 4)
plt.title("b. Lightness (Grayscale)")
plt.imshow(img_lightness, cmap='gray', vmin=0, vmax=255)

# --- c. Luminance ---
plt.subplot(3, 2, 5)
plt.title("c. Luminance (Original)")
plt.imshow(img_rgb)

plt.subplot(3, 2, 6)
plt.title("c. Luminance (Grayscale)")
plt.imshow(img_luminance, cmap='gray', vmin=0, vmax=255)

plt.tight_layout()
plt.show()
```

	a. Averaging (Original)  b. Lightness (Original)  c. Luminance (Original)  a. Averaging (Grayscale)  b. Lightness (Grayscale)  c. Luminance (Grayscale) 
5	Tampilkanlah warna tertentu pada citra, dan ubah warna lain menjadi grayscale. Misal, tampilkan warna merah pada citra masukan dan ubah bagian lain yang tidak berwarna merah menjadi grayscale seperti pada contoh berikut:



```
# 1. Load gambar
img_bgr = cv.imread('/content/drive/MyDrive/PCVK 2026/Gambar/buah.png')
img_rgb = cv.cvtColor(img_bgr, cv.COLOR_BGR2RGB)

# 2. Konversi citra dari RGB ke HSV
img_hsv = cv.cvtColor(img_rgb, cv.COLOR_RGB2HSV)

# 3. Tentukan rentang batas warna merah pada ruang HSV
# Karena warna merah berada di dua ujung rentang (0-10 dan 170-180), kita buat dua mask lalu digabungkan
lower_red1 = np.array([0, 100, 100])
upper_red1 = np.array([10, 255, 255])
mask1 = cv.inRange(img_hsv, lower_red1, upper_red1)

lower_red2 = np.array([170, 100, 100])
upper_red2 = np.array([180, 255, 255])
mask2 = cv.inRange(img_hsv, lower_red2, upper_red2)

# Gabungkan mask merah pertama dan kedua
mask_red = cv.bitwise_or(mask1, mask2)

# 4. Ubah seluruh citra asli menjadi grayscale menggunakan metode luminance
gray_single = 0.299 * img_rgb[:, :, 0] + 0.587 * img_rgb[:, :, 1] + 0.114 * img_rgb[:, :, 2]
# Konversi kembali ke 3 channel (RGB) agar ukurannya kompatibel dengan citra asli
img_gray_3ch = np.stack((gray_single, gray_single, gray_single), axis=-1).astype(np.uint8)

# 5. Gabungkan bagian warna merah asli dengan latar belakang yang sudah berformat grayscale
# Ambil bagian merah dari citra asli menggunakan mask
red_extracted = cv.bitwise_and(img_rgb, img_rgb, mask=mask_red)

# Ambil bagian latar belakang (selain merah) dari citra grayscale
inverse_mask = cv.bitwise_not(mask_red)
background_gray = cv.bitwise_and(img_gray_3ch, img_gray_3ch, mask=inverse_mask)

# Ambil bagian latar belakang (selain merah) dari citra grayscale
inverse_mask = cv.bitwise_not(mask_red)
background_gray = cv.bitwise_and(img_gray_3ch, img_gray_3ch, mask=inverse_mask)

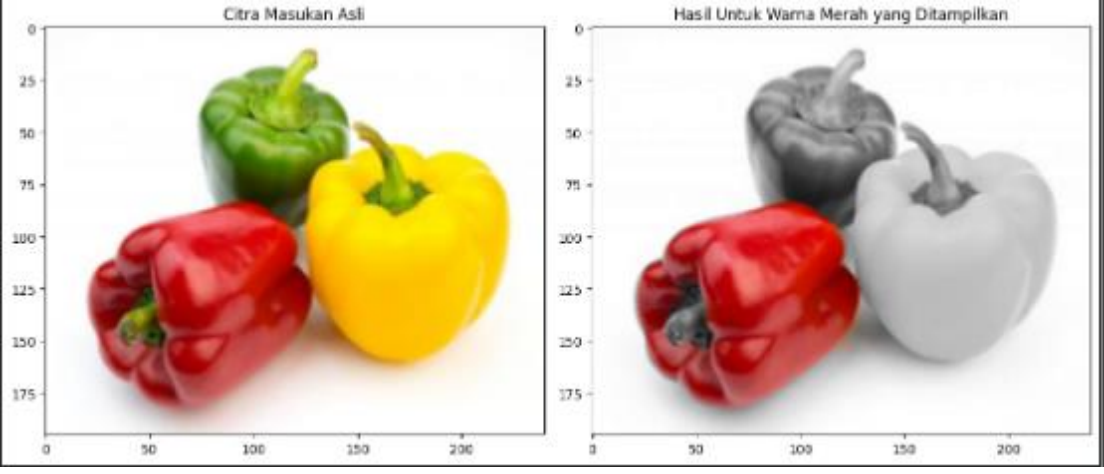
# Hasil akhir adalah gabungan keduanya
final_isolated = cv.add(red_extracted, background_gray)

# 6. Tampilkan hasil menggunakan Matplotlib
plt.figure(figsize=(12, 6))

plt.subplot(1, 2, 1)
plt.title("Citra Asli")
plt.imshow(img_rgb)

plt.subplot(1, 2, 2)
plt.title("Hasil Untuk Warna Merah yang Ditampilkan")
plt.imshow(final_isolated)

plt.tight_layout()
plt.show()
```

	
6	Buat Gamma Correction sesuai dengan petunjuk berikut Percobaan ini akan meminta anda membuat Gamma Correction. Pada percobaan ini, nilai Gamma akan diset dengan meminta masukan dari pengguna. Berikut adalah kode untuk meminta masukan nilai dari pengguna. Lanjutkan kode tersebut dengan membuat image dengan gamma correction sesuai rumus yang telah diberikan <pre> print(' Gamma Correction pada citra ') print('-----') try: gamma = float(input('Masukkan nilai Gamma: ')) except ValueError: print('Error, not a number') # Load gambar original = cv.imread('/content/drive/MyDrive/PCVK 2026/Gambar/rumah.jpeg') # Konversi BGR ke RGB original_rgb = cv.cvtColor(original, cv.COLOR_BGR2RGB) # Menerapkan formula Gamma Correction normalized_image = original_rgb / 255.0 gamma_image = np.array(255 * (normalized_image ** gamma), dtype=np.uint8) # Menampilkan hasil citra asli dan hasil gamma correction secara berdampingan plt.figure(figsize=(12, 6)) plt.subplot(1, 2, 1) plt.title("Citra Asli") plt.imshow(original_rgb) plt.subplot(1, 2, 2) plt.title(f"Gamma Correction (γ = {gamma})") plt.imshow(gamma_image) plt.tight_layout() plt.show </pre>



	*** Gamma Correction pada citra ----- Masukkan nilai Gamma: 0.5 
7	Buat Simulasi Image Depth Percobaan ini digunakan sebagai simulasi dari proses kuantisasi citra. Pada kuantisasi citra, pixel dapat direpresentasikan dengan n-bit kedalaman (default menggunakan 8-bit). Pada pixel 8-bit, warna yang memungkinkan adalah 256 warna, dari 0 (0000 0000) hingga 255 (1111 1111). Pada pixel 7-bit, warna yang memungkinkan adalah 128 warna, dari 0 (000 0000) hingga 127 (111 1111). Kemungkinan warna didapat dari pangkat 2 jumlah bit. Jika 7bit, maka jumlah warnanya adalah $2^7 = 128$, dst. Berikut adalah kode untuk membaca citra masukan dan memberi nilai kedalaman citra, silahkan lanjutkan kode program berikut sehingga menghasilkan keluaran seperti contoh pada Gambar.



```
try:
    # Masukkan bit depth dari kamu
    input_bit = int(input('Masukkan bit depth tujuan (1-8): '))
except ValueError:
    print('Error, not a number')

bit_depth = input_bit
level = 255 / (pow(2, bit_depth) - 1)
original = cv.imread('/content/drive/MyDrive/PCVK 2026/Gambar/buah.png', cv.IMREAD_GRAYSCALE)
depth_image = np.zeros(original.shape, original.dtype)

# Proses kuantisasi citra per piksel
for y in range(original.shape[0]):
    for x in range(original.shape[1]):
        val = np.floor(original[y, x] / level) * level
        depth_image[y, x] = np.uint8(val)

# Menampilkan informasi teks sesuai contoh
print(f"Bit depth awal : 8 bit")
print(f"Bit depth tujuan: {bit_depth} bit")
print(f"Jumlah level : {int(pow(2, bit_depth))}")

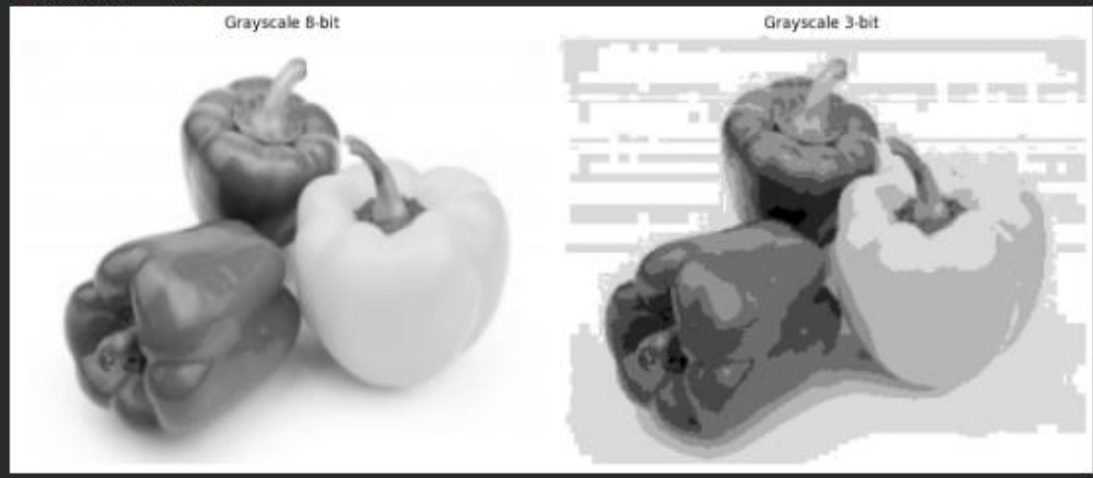
# Menampilkan hasil citra Grayscale 8-bit dan Grayscale n-bit secara berdampingan
plt.figure(figsize=(12, 6))

plt.subplot(1, 2, 1)
plt.title("Grayscale 8-bit")
plt.imshow(original, cmap='gray', vmin=0, vmax=255)
plt.axis('off')
```

```
plt.subplot(1, 2, 2)
plt.title(f"Grayscale {bit_depth}-bit")
plt.imshow(depth_image, cmap='gray', vmin=0, vmax=255)
plt.axis('off')

plt.tight_layout()
plt.show()
```

```
Masukkan bit depth tujuan (1-8): 3
Bit depth awal : 8 bit
Bit depth tujuan: 3 bit
Jumlah level : 8
```



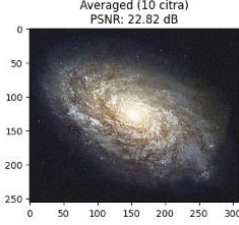
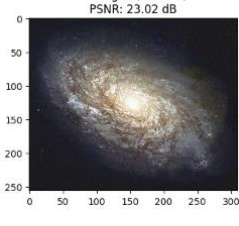
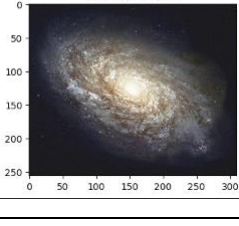
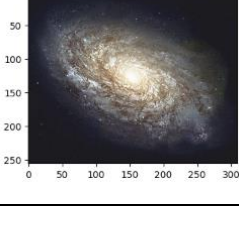


Buat modul average denoising sesuai dengan rumus yang telah diberikan pada sub bab sebelumnya.

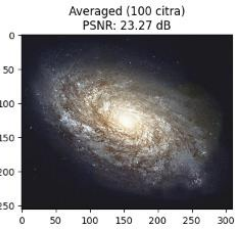
Citra asli sudah disediakan pada **/images/galaxy.jpg**.

100 Citra dengan Gaussian Noise sudah disediakan pada **/images/noises/*.jpg**

Anda dapat menggunakan code berikut untuk membaca semua image dalam satu folder , gunakan modul glob (**import glob**)

No	Jumlah Citra Average	Citra Asli	Image PSNR (dB)	Nilai PSNR (dB)
1	10		22.82	
2	20		23.02	
3	40		23.12	
4	80		23.17	



5	100		23.27
---	-----	---	-------

```
import matplotlib.pyplot as plt
from math import log10, sqrt

# Membaca Citra Asli
original_path = '/content/drive/MyDrive/PCVK 2026/Gambar/noises/galaxy.jpg'
img_ori = cv.imread(original_path)

# Memuat 100 citra noise secara berurutan (1.jpg sampai 100.jpg)
cv_img = []
for i in range(1, 101):
    img_path = f'/content/drive/MyDrive/PCVK 2026/Gambar/noises/{i}.jpg'
    n = cv.imread(img_path)
    if n is not None:
        cv_img.append(n)

print(f"Total citra noise yang berhasil dimuat: {len(cv_img)}")

# Daftar jumlah citra yang akan di-average sesuai tabel tugas
jumlah_list = [10, 20, 40, 80, 100]

# Fungsi untuk menghitung nilai PSNR
def hitung_psnr(img1, img2):
    mse = np.mean((img1.astype(np.float32) - img2.astype(np.float32)) ** 2)
    if mse == 0:
        return 100
    max_pixel = 255.0
    psnr = 20 * log10(max_pixel / sqrt(mse))
    return psnr
```



```
# Proses Averaging dan Perhitungan PSNR
for jml in jumlah_list:
    if len(cv_img) < jml:
        print(f"Jumlah citra kurang dari {jml}!")
        continue

    # Ambil subset citra sebanyak 'jml' (dari 1 sampai jml)
    subset_imgs = cv_img[:jml]

    # Inisialisasi accumulator dengan tipe float untuk menghindari overflow
    accumulated = np.zeros_like(subset_imgs[0], dtype=np.float32)
    for img in subset_imgs:
        accumulated += img.astype(np.float32)

    # Hitung rata-rata
    averaged_img = (accumulated / jml).astype(np.uint8)

    # Hitung nilai PSNR terhadap citra asli
    psnr_val = hitung_psnr(img_ori, averaged_img)
    print(f"Jumlah Citra di-Average: {jml:3d} | Nilai PSNR: {psnr_val:.4f} dB")

    # Tampilkan hasil gambar
    plt.figure(figsize=(4, 4))
    plt.title(f"Averaged ({jml} citra)\nPSNR: {psnr_val:.2f} dB")
    plt.imshow(cv.cvtColor(averaged_img, cv.COLOR_BGR2RGB))
    plt.show()
```

Apakah peningkatan jumlah citra selalu memberikan peningkatan PSNR yang signifikan?

Jawaban:

Tidak selalu. Meskipun jumlah citra yang digabungkan ditambah, kenaikan nilai PSNR-nya lama-kelamaan akan semakin kecil (melandai). Jadi, penambahan jumlah gambar tidak membuat kualitasnya melonjak secara drastis terus-menerus.

Pada jumlah citra berapa peningkatan mulai tidak terlalu signifikan? Apa Kesimpulan yang Anda peroleh dari percobaan ini?

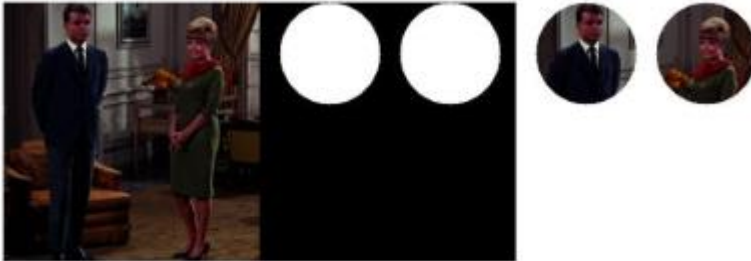
Jawaban:

- Mulai tidak signifikan pada citra ke 40 ke atas dan peningkatan mulai terasa sangat kecil.
- Kesimpulan:

Teknik rata-rata terbukti ampuh untuk menghilangkan noise pada gambar galaksi dan menambah gambar sampai 100 buah ternyata kurang efektif dari segi waktu proses komputer, karena kualitas gambar di angka 40-an sudah hampir mirip hasilnya dengan di angka 100. Jadi, sekitar 40 citra saja sudah cukup optimal!

9. Buat image masking

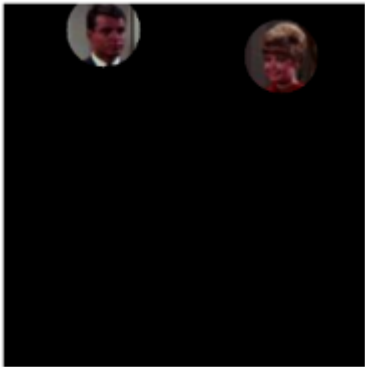
Buat image masking untuk image berikut. Image kiri adalah image asli (images/couple.tiff), sedangkan image paling kanan adalah hasilnya:



Lakukan percobaan menggunakan operator lain dan tunjukkan hasilnya pada modul ini. Tuliskan hasil analisa anda kenapa citra keluarannya seperti itu

No	Operator	Image Input	Image Output
1	NOT (komplemen)		
2	OR (atau)		



3	AND (dan)		
4	NAND (Not And)		
5	XOR (Exclusive Or)		



```
# Load citra asli
img_path = '/content/drive/MyDrive/PCVK 2026/Gambar/pasangan.png'
img = cv.imread(img_path)
img_rgb = cv.cvtColor(img, cv.COLOR_BGR2RGB)
h, w, _ = img.shape

# Buat mask lingkaran untuk wajah pria dan wanita
mask = np.zeros((h, w), dtype=np.uint8)
cv.circle(mask, (95, 25), 35, 255, -1) # Wajah pria di sebelah kiri
cv.circle(mask, (265, 50), 35, 255, -1) # Wajah wanita di sebelah kanan

# Konversi mask ke 3 channel
mask_3ch = cv.cvtColor(mask, cv.COLOR_GRAY2RGB)

# 1. Operator AND (Masking wajah)
res_and = cv.bitwise_and(img_rgb, mask_3ch)

# 2. Operator OR (Menggabungkan citra asli dengan mask)
res_or = cv.bitwise_or(img_rgb, mask_3ch)

# 3. Operator NOT (Kebalikan mask)
inv_mask_3ch = cv.cvtColor(cv.bitwise_not(mask), cv.COLOR_GRAY2RGB)
res_not = cv.bitwise_and(img_rgb, inv_mask_3ch)

# 4. Operator NAND (Negasi dari hasil AND)
res_nand = cv.bitwise_not(res_and)

# 5. Operator XOR (Exclusive OR antara citra asli dan mask)
res_xor = cv.bitwise_xor(img_rgb, mask_3ch)

# Menampilkan Hasil Visualisasi dalam Grid 3x2
plt.figure(figsize=(12, 12))
```

```
# Menampilkan Hasil Visualisasi dalam Grid 3x2
plt.figure(figsize=(12, 12))

plt.subplot(3, 2, 1)
plt.title("Citra Asli")
plt.imshow(img_rgb)
plt.axis('off')

plt.subplot(3, 2, 2)
plt.title("Mask (Area Wajah)")
plt.imshow(mask, cmap='gray')
plt.axis('off')

plt.subplot(3, 2, 3)
plt.title("Hasil AND")
plt.imshow(res_and)
plt.axis('off')

plt.subplot(3, 2, 4)
plt.title("Hasil OR")
plt.imshow(res_or)
plt.axis('off')

plt.subplot(3, 2, 5)
plt.title("Hasil NOT")
plt.imshow(res_not)
plt.axis('off')
```



```
# Menampilkan NAND dan XOR di baris tambahan
plt.figure(figsize=(12, 6))

plt.subplot(1, 2, 1)
plt.title("Hasil NAND")
plt.imshow(res_nand)
plt.axis('off')

plt.subplot(1, 2, 2)
plt.title("Hasil XOR")
plt.imshow(res_xor)
plt.axis('off')

plt.tight_layout()
plt.show()
```

Tuliskan hasil analisa anda:

1. NOT: Membalikkan seluruh nilai piksel. Area wajah yang awalnya putih pada mask berubah menjadi hitam, sehingga bagian wajah tertutup lingkaran hitam. Sebaliknya, area luar wajah menjadi kebalikan dari latar belakang aslinya.
2. OR: Menggabungkan citra dengan mask karena menggunakan operasi gabungan, area lingkaran mask yang bernilai putih memenuhi bagian wajah, sehingga area wajah pada citra tertutup oleh bulatan putih penuh.
3. AND: Berfungsi sebagai masking utama. Hanya area yang beririsan antara bagian putih mask dan citra asli saja yang ditampilkan, sedangkan area di luar lingkaran menjadi hitam penuh.
4. NAND: Area wajah yang sebelumnya dipotong oleh operasi AND kini dibalik menjadi warna negatif dan area latar belakang sekitarnya berubah menjadi warna putih terang.
5. XOR: Area wajah mengalami perubahan warna menjadi negative, sementara bagian tubuh dan latar belakang ruangan di luar area wajah tetap ditampilkan secara normal seperti citra aslinya.

10. Foto Malam Hari

Carilah foto malam hari yang gelap. Namun, beberapa bagian gambar memiliki detail yang sebenarnya cukup baik. Pilih salah satu metode berikut dan implementasikan pada foto tersebut (Linear Brightness, Contrast, Logarithmic Brightness, Gamma Correction). Sebutkan alasan dan resiko metode yang Anda gunakan.

```
# Load citra
img_path = '/content/drive/MyDrive/PCVK 2026/Gambar/malam hari.jpeg'
img = cv.imread(img_path)
img_rgb = cv.cvtColor(img, cv.COLOR_BGR2RGB)

# 2. Terapkan Gamma Correction
gamma = 0.4 # Nilai < 1 mencerahkan bagian gelap, > 1 menggelapkan

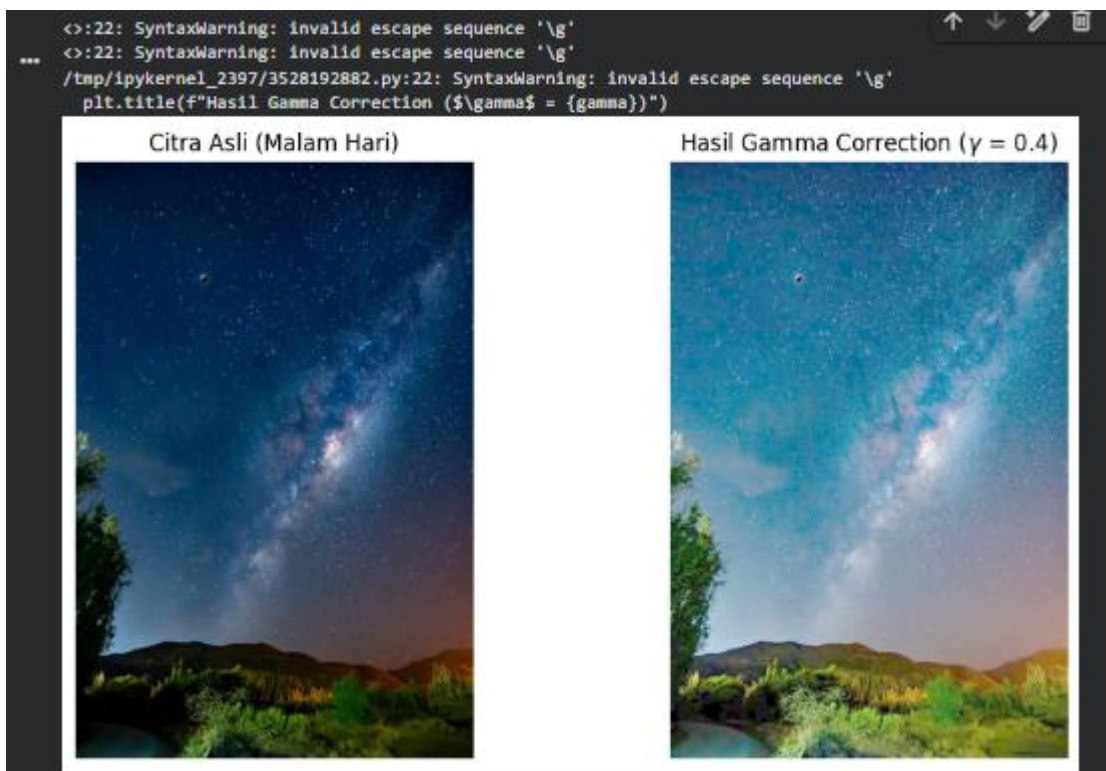
# Normalisasi piksel ke rentang [0, 1]
normalized = img_rgb / 255.0
gamma_corrected = np.array(255 * (normalized ** gamma), dtype=np.uint8)

# Menampilkan Hasil Visualisasi
plt.figure(figsize=(10, 5))

plt.subplot(1, 2, 1)
plt.title("Citra Asli (Malam Hari)")
plt.imshow(img_rgb)
plt.axis('off')

plt.subplot(1, 2, 2)
plt.title(f"Hasil Gamma Correction ($\gamma$ = {gamma})")
plt.imshow(gamma_corrected)
plt.axis('off')

plt.tight_layout()
plt.show()
```



Metode yang Dipilih: Gamma Correction (dengan nilai $\gamma < 1$).

Alasan Menggunakan Metode Ini:



Gamma correction menggunakan fungsi non-linier yang sangat efektif untuk mencerahkan bagian yang gelap tanpa merusak area yang sudah terang (seperti lampu kota atau bintang). Hal ini membuat detail tersembunyi pada area gelap menjadi lebih tampak jelas dan nyaman dilihat oleh mata manusia.

Risiko Metode Ini:

Jika nilai γ diset terlalu rendah (terlalu kecil dari 1), gambar akan menjadi terlalu terang secara berlebihan dan munculnya efek noise yang sebelumnya tak terlihat pada area gelap akan ikut terekspos dan menjadi lebih jelas setelah diterangkan.

11. Buka gambar crayfish.jpg pada folder Images.



Perbaiki kualitas pada citra tersebut dengan metode operasi dasar citra apapun. Diperbolehkan menggunakan lebih dari 1 metode. Kemudian jelaskan:

- Metode yang dipilih : Grayscale
- Alasan pemilihan : Karena airnya keruh dan warnanya kusam, warna asli gambar tidak terlalu membantu. Mengubahnya jadi hitam-putih bikin gambar lebih simpel dan fokus ke bentuk asli lobster karangnya.
- Menentukan parameter terbaik : Memakai fungsi standar OpenCV yaitu `cv.COLOR_BGR2GRAY`
- Menghasilkan citra dengan menunjukkan before-after



```
# Load citra
img_path = '/content/drive/MyDrive/PCVK 2026/Gambar/crayfish.jpg'
img = cv.imread(img_path)
img_rgb = cv.cvtColor(img, cv.COLOR_BGR2RGB)

# Konversi citra ke Grayscale
gray = cv.cvtColor(img, cv.COLOR_BGR2GRAY)

# Menampilkan Hasil Visualisasi (Before-After)
plt.figure(figsize=(12, 6))

plt.subplot(1, 2, 1)
plt.title("Citra Asli (RGB)")
plt.imshow(img_rgb)
plt.axis('off')

plt.subplot(1, 2, 2)
plt.title("Hasil Grayscale (After)")
plt.imshow(gray, cmap='gray')
plt.axis('off')

plt.tight_layout()
plt.show()
```

Citra Asli (RGB)



Hasil Grayscale (After)

